UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE - UFRN
CENTRO DE ENSINO SUPERIOR DO SERIDÓ - CERES
DEPARTAMENTO DE COMPUTAÇÃO E TECNOLOGIA DCT

CERES 50 ANOS

FELIPE EDUARDO SILVA FERNANDES
Matrícula: 20250037964

RELATÓRIO TÉCNICO: ANÁLISE EXPLORATÓRIA, PREPARAÇÃO E AVALIAÇÃO COMPARATIVA DE MODELOS DE IA

Link do Dataset:
`https://www.kaggle.com/datasets/razanihababdellatif/food-delivery-orders-and-eta-logistics-dataset`

Repositório GitHub:
`https://github.com/Felipe-edu34/food-delivery-eta-analysis-ml.git`

Caicó - RN
2026

FELIPE EDUARDO SILVA FERNANDES
Matrícula: 20250037964

RELATÓRIO TÉCNICO: ANÁLISE EXPLORATÓRIA, PREPARAÇÃO E AVALIAÇÃO COMPARATIVA DE MODELOS DE IA

Relatório técnico de Atividade Prática Avaliativa apresentado como requisito para obtenção de nota na disciplina **DCT1401 - Inteligência Artificial**, ofertada pela UFRN no período letivo 2026.2.

Professor: Prof. Thommas K. S. Flores

Universidade Federal do Rio Grande do Norte - UFRN
Centro de Ensino Superior do Seridó - CERES
Departamento de Computação e Tecnologia - DCT

Caicó - RN
2026

Sumário

1 Introdução **5**

2 Análise Exploratória (EDA) e Pré-processamento **5**
2.1 Inspeção da Base Bruta e Diagnóstico de Valores Ausentes 5
2.2 Estatísticas Descritivas e Tratamento de Outliers por Winsorization 6
2.3 Eliminação de Multicolinearidade e Vazamento de Dados (Data Leakage) . 8
2.4 Seleção Estatística de Features para Regressão e Classificação 9

3 Protocolo Experimental e Ambiente Computacional **10**

4 Modelo 1: Sistema Especialista (Regras SE-ENTÃO) **10**
4.1 Base de Conhecimento e Motor de Inferência 10
4.2 Cronometragem do Esforço Humano e Profiling na Inferência 11

5 Modelo 2: Árvore de Decisão **12**
5.1 Entropia, Ganho de Informação, Critérios de Impureza e Overfitting 12
5.2 Visualização da Árvore e Interpretação de 3 Regras Extraídas 14

6 Modelo 3: Ensemble Learning (Random Forest e Gradient Boosting) **15**
6.1 Ajuste de Hiperparâmetros e Curva de Erro por Número de Árvores 15
6.2 Importância das Features vs. Regras do Sistema Especialista 16

7 Resultados: Desempenho, Generalização e Eficiência Computacional **17**
7.1 Métricas de Desempenho no Conjunto de Teste (Aula 08) 17
7.2 Análise de Generalização e Validação Cruzada (K = 5) 20
7.3 Eficiência Computacional e Tabela-Resumo Final 21

8 Análise Comparativa e Discussão **22**

9 Considerações Finais **23**

10 Referência ao Repositório **23**

Lista de Figuras

1 Concentração de valores ausentes nas 5 colunas afetadas da base bruta ($N = 50.000$). 6
2 Painel de boxplots identificando dispersão e outliers pelo método IQR nas variáveis numéricas. 7
3 Efeito do tratamento de outliers por Winsorization em `actual_delivery_time_minutes` (teto de 120 min). 7
4 Matriz de Correlação de Pearson das variáveis numéricas da base. 8
5 Ranking das 10 variáveis mais relevantes para Regressão por Teste F (`f_regression`) e $|r|$ de Pearson. 9
6 Preservação da proporção das classes no split estratificado e ranking ANOVA F para Classificação. 9
7 Desempenho do Sistema Especialista no teste: Matriz de Confusão, estabilidade Treino vs. Teste, regressão em patamares e cobertura por regra. . 12
8 Esforço humano cronometrado por sessão e tempo médio por regra (Aula 05). 12
9 Ganho de Informação $G(D, A)$ e Redução de Gini $\Delta\text{Gini}(D, A)$ para partições candidatas no nó raiz. 13
10 Comparação empírica dos critérios de impureza na Classificação e na Regressão. 13
11 Curva de complexidade e poda (`max_depth`) evidenciando o gap de overfitting. 14
12 Primeiros 3 níveis da Árvore de Decisão ajustada para Classificação (profundidade 12, 203 nós). 15
13 Curva de erro da Random Forest em função do número de árvores $B \in [1, 200]$ (Aula 07). 16
14 Importância das features (`feature_importances_`) na Random Forest e no Gradient Boosting. 17
15 Matrizes de Confusão dos 4 modelos no conjunto de teste ($N = 9.827$). . . 18
16 Curvas ROC comparativas no conjunto de teste. 18
17 Curvas Precision-Recall (PR) comparativas no conjunto de teste. 19
18 Valores Reais vs. Valores Previstos na Regressão para os 4 modelos no teste. 19
19 Gráfico de resíduos ($e_i = y_i - \hat{y}_i$) dos 4 modelos de Regressão no teste. . . . 20
20 Desempenho no Treino vs. Teste e Gap de Generalização para todos os modelos. 21
21 Estabilidade dos modelos nos 5 folds da Validação Cruzada ($K = 5$). . . . 21
22 Fronteira de Pareto entre Custo de Inferência, Tamanho em Disco e Qualidade Preditiva. 22

Lista de Tabelas

1 Registro das sessões cronometradas de elaboração das regras SE-ENTÃO. . 11
2 Comparação entre a Árvore Sem Restrição e a Árvore Ajustada (evidência de overfitting). 14
3 Tabela-Resumo Final Consolidada: Desempenho no Teste, Generalização e Eficiência Computacional ($N_{\text{teste}} = 9.827$). 21

1 Introdução

Quem já pediu comida por aplicativo em noite de chuva sabe que a estimativa de tempo que aparece na tela nem sempre bate com a realidade. Do ponto de vista de engenharia, prever exatamente em quantos minutos um pedido vai chegar e antecipar se ele vai atrasar ou não são dois problemas clássicos de logística urbana. Neste trabalho, atuo como engenheiro de Machine Learning analisando a base *Food Delivery Orders and ETA Logistics Dataset* para responder a uma demanda direta de consultoria: qual abordagem de modelagem é mais adequada para a operação do cliente, considerando qualidade preditiva, capacidade de generalização, esforço humano de construção e eficiência computacional?

Para responder a isso com base em evidências, estruturei o trabalho começando pela limpeza rigorosa da base bruta — que conta com exatos **50.000 registros** distribuídos em **37 colunas** — até a construção e avaliação comparativa das três famílias de modelos estudadas nas Aulas 05, 06 e 07, avaliadas segundo as métricas da Aula 08:

1. **Problema de Regressão (A promessa ao cliente):** prever o tempo real de entrega em minutos (`actual_delivery_time_minutes`), permitindo que o aplicativo forneça estimativas confiáveis.
2. **Problema de Classificação (A operação preventiva):** prever a ocorrência de atraso na entrega (`late_delivery`, onde 0 = no prazo e 1 = atrasado), permitindo acionar alertas preventivos antes que a falha gere reclamações.

Além de implementar e medir o **Sistema Especialista baseado em regras SE-ENTÃO**, a **Árvore de Decisão** e os métodos de **Ensemble Learning** (*Random Forest* e *Gradient Boosting*), revisei toda a etapa exploratória e de pré-processamento da primeira unidade para corrigir armadilhas de vazamento de dados (*data leakage*) e multicolinearidade.

2 Análise Exploratória (EDA) e Pré-processamento

2.1 Inspeção da Base Bruta e Diagnóstico de Valores Ausentes

Carregando a base original (`50.000 linhas x 37 colunas`) com o `pandas`, identifiquei 20 variáveis numéricas (12 do tipo `float64` e 8 do tipo `int64`), 15 variáveis categóricas/identificadores textuais e 2 marcadores de data/hora (`order_timestamp` e `order_date`). A inspeção das primeiras e últimas linhas e dos limites mínimos mostrou que não existem valores negativos inválidos nas grandezas físicas (distância mínima de 0,50 km, tempo de preparo mínimo de 5,0 min e tempo real mínimo de 8,5 min).

O levantamento de valores nulos revelou que apenas 5 colunas possuíam dados ausentes, e nenhuma delas ocorria de forma aleatória:

- `cancellation_reason`: ausente em **49.132 linhas** (98,26% da base), pois só é preenchida quando o pedido sofre cancelamento.

- `actual_delivery_time_minutes`, `late_delivery`, `tip_amount` e `customer_rating`: exatamente **868 valores ausentes** (1,74% da base) nas mesmas linhas.

Cruzando esses 868 registros com a coluna `order_status`, confirmei que 100% deles são pedidos com status `Cancelled`. Como um pedido cancelado não teve entrega física, imputar valores nos alvos (`actual_delivery_time_minutes` ou `late_delivery`) inventaria entregas fictícias. Por isso, filtrei a base mantendo apenas os **49.132 pedidos concluídos** (`order_status == 'Completed'`) e descartei a coluna `cancellation_reason`. A Figura 1 mostra a concentração desses valores ausentes na base bruta.

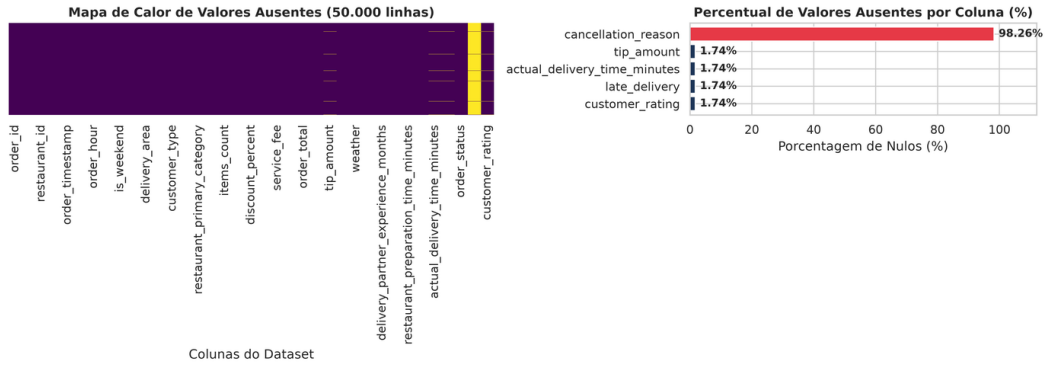


Figura 1: Concentração de valores ausentes nas 5 colunas afetadas da base bruta ($N = 50.000$).

2.2 Estatísticas Descritivas e Tratamento de Outliers por Winsorization

Sobre os 49.132 pedidos entregues, o tempo real de entrega (`actual_delivery_time_minutes`) apresenta média de 44,83 min ($\pm$14,84 min) e mediana de 43,50 min, com leve assimetria positiva (0,58). A distância média é de 5,00 km (mediana 4,49 km, chegando a 40,0 km) e o tempo médio de preparo no restaurante é de 17,29 min.

Aplicando o método formal do Intervalo Interquartil (IQR $= Q_3 - Q_1$, com limites $L_{\text{inf}} = Q_1 - 1{,}5 \cdot \text{IQR}$ e $L_{\text{sup}} = Q_3 + 1{,}5 \cdot \text{IQR}$), mapeei os valores extremos das variáveis contínuas (Figura 2).

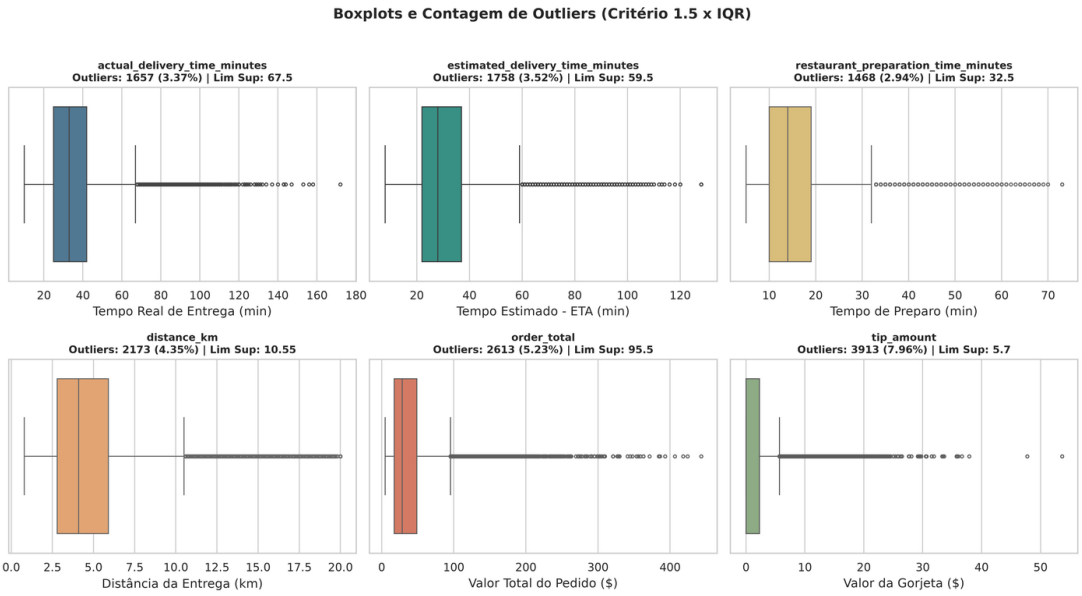


Figura 2: Painel de boxplots identificando dispersão e outliers pelo método IQR nas variáveis numéricas.

Variáveis como `order_total` (1.874 casos acima de \$98,05) e `distance_km` (449 casos acima de 13,05 km) refletem compras grandes de fim de semana e rotas longas reais; removê-las cegaria o modelo para entregas distantes. Já no alvo `actual_delivery_time_minutes`, havia um grupo isolado de **25 pedidos** (0,05% da base) que ultrapassava 2 horas de duração (chegando a 146,2 min), causado por esquecimento de encerramento da corrida no aplicativo. Para evitar que essas 25 anomalias distorcessem o erro quadrático (MSE/RMSE), apliquei *Winsorization* (truncamento superior) em 120,0 min:

$$y_i^{\text{tratado}} = \min(y_i,\ 120{,}0) \tag{1}$$

A Figura 3 compara a distribuição antes e depois do truncamento, mostrando que a estrutura da distribuição foi preservada.

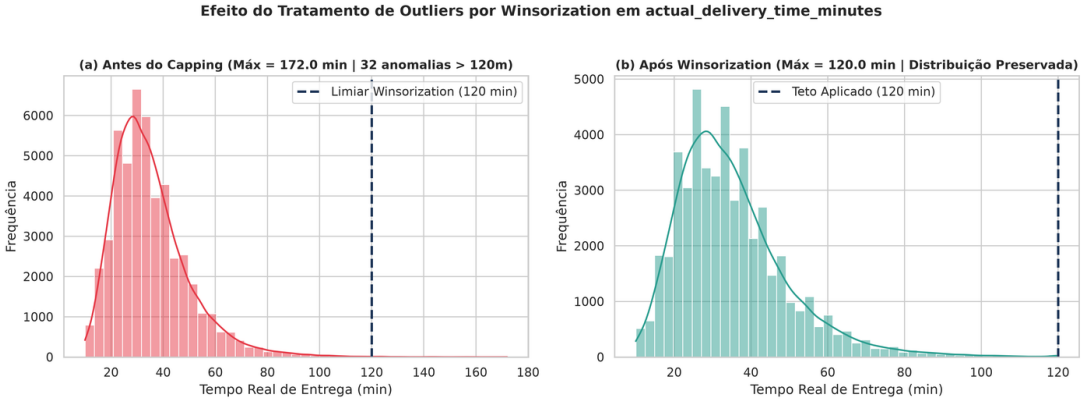


Figura 3: Efeito do tratamento de outliers por Winsorization em `actual_delivery_time_minutes` (teto de 120 min).

2.3 Eliminação de Multicolinearidade e Vazamento de Dados (Data Leakage)

A análise da matriz de correlação de Pearson (Figura 4) permitiu corrigir dois problemas estruturais antes da modelagem:

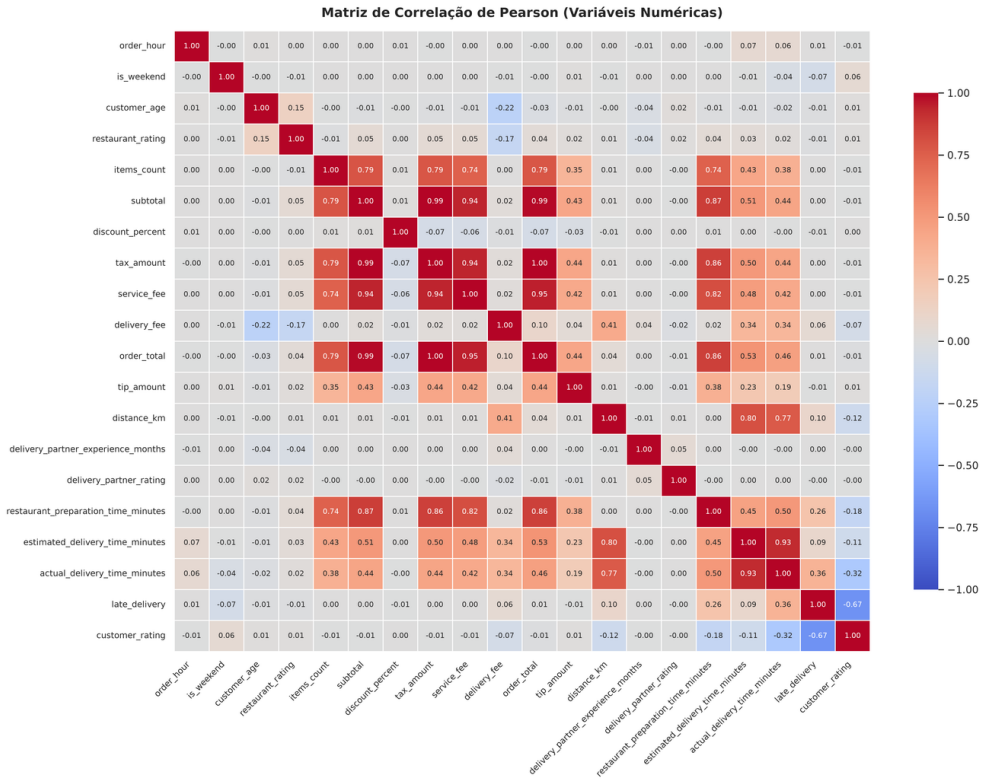


Figura 4: Matriz de Correlação de Pearson das variáveis numéricas da base.

1. **Multicolinearidade no bloco financeiro:** as colunas `subtotal`, `tax_amount`, `service_fee` e `order_total` apresentavam correlações entre $r = 0{,}9745$ e $r = 1{,}0000$, pois impostos e taxas são frações fixas do subtotal. Descartei `subtotal`, `tax_amount` e `service_fee`, mantendo apenas `order_total` e `items_count`.

2. **Prevenção de Data Leakage pós-entrega:** a coluna `customer_rating` possui correlação de $r = -0{,}6745$ com `late_delivery` porque clientes insatisfeitos atribuem nota baixa *após* o pedido atrasar. Da mesma forma, `tip_amount` pode ser alterada na conclusão do pedido. Ambas foram removidas, assim como `late_delivery` foi excluída da regressão e `actual_delivery_time_minutes` foi excluída da classificação. Já a estimativa inicial do aplicativo (`estimated_delivery_time_minutes`, $r = 0{,}9298$) é gerada na abertura do pedido (pré-entrega) e foi mantida como preditor base legítimo.

Após remover colunas de identificação (`order_id`, `customer_id`, `restaurant_id`, `driver_id`, datas, cidade e pagamento) e aplicar *One-Hot Encoding* (`drop_first=True`) nas categóricas nominais (`weather`, `traffic_level`, `customer_type` e `day_of_week`), obtivemos **26 features preditivas** limpas.

2.4 Seleção Estatística de Features para Regressão e Classificação

Calculando os testes estatísticos exclusivamente sobre o conjunto de treino:

- **Regressão (`f_regression` e $|r|$ de Pearson):** como mostra a Figura 5, o poder preditivo do tempo total de entrega é dominado por `estimated_delivery_time_minutes` ($F = 251.116{,}8$, $r = 0{,}9299$), `distance_km` ($F = 57.289{,}7$, $r = 0{,}7702$), `restaurant_preparation_time_minutes` ($F = 13.314{,}8$, $r = 0{,}5032$) e `order_total` ($r = 0{,}4568$).
- **Classificação (ANOVA F e Qui-Quadrado χ^2):** a variável alvo natural `late_delivery` possui **63,43% de entregas no prazo (classe 0)** e **36,57% de atrasos (classe 1)**. Conforme ilustrado na Figura 6, enquanto a distância lidera na regressão, a ocorrência de atraso é fortemente governada pelo clima adverso (`weather_Rain` com $F = 4.664{,}3$ e $\chi^2 = 3.553{,}3$; `weather_Storm` com $F = 3.575{,}6$ e $\chi^2 = 3.164{,}7$) e pelo gargalo na cozinha (`restaurant_preparation_time_minutes`, $F = 2.662{,}5$).

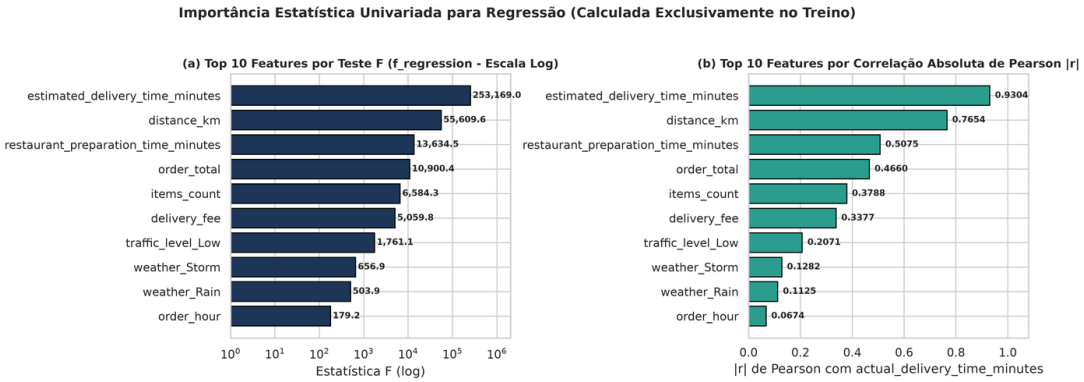


Figura 5: Ranking das 10 variáveis mais relevantes para Regressão por Teste F (`f_regression`) e $|r|$ de Pearson.

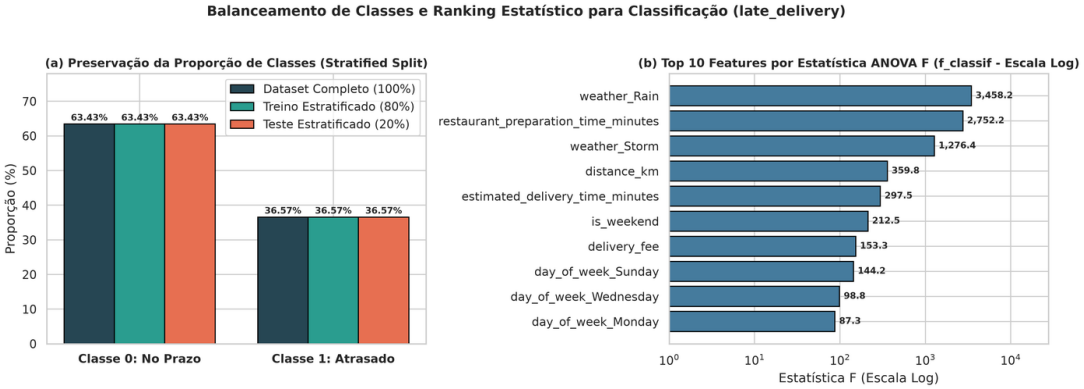


Figura 6: Preservação da proporção das classes no split estratificado e ranking ANOVA F para Classificação.

3 Protocolo Experimental e Ambiente Computacional

Todos os experimentos nos notebooks `03` a `08` seguiram exatamente o mesmo protocolo:

- **Particionamento 80% / 20%:** as 49.132 entregas foram divididas em **39.305 amostras de treino** e **9.827 amostras de teste** com semente fixa (`random_state = 42`) e amostragem estratificada (`stratify=y`) na classificação.
- **Validação Cruzada e Prevenção de Leakage:** o conjunto de teste foi usado apenas na avaliação final. O ajuste de hiperparâmetros utilizou validação cruzada de 5 pastas ($K = 5$) dentro do treino, e a padronização (`StandardScaler`) foi ajustada apenas sobre `X_train` via `Pipeline`.
- **Especificações da Máquina: Notebook Lenovo** equipado com processador **12th Gen Intel Core i7-1255U** (12 threads, TDP considerado de 45 W), **12 GB de memória RAM** ($\approx$ 10 GiB livres ao sistema operacional após reserva de vídeo integrado), armazenamento **SSD de 512 GB**, sistema operacional **Linux (Kernel 7.0.0-38-generic)**, **Python 3.14.4** e `scikit-learn 1.9.0`.

4 Modelo 1: Sistema Especialista (Regras SE-ENTÃO)

4.1 Base de Conhecimento e Motor de Inferência

Seguindo a Aula 05, implementei o Sistema Especialista em `src/expert_system.py` separando a **Base de Conhecimento** (`rules/regras_classificacao.txt` e `rules/regras_regressao.txt`) do **Motor de Inferência** (`ForwardChainingEngine`). Utilizando exclusivamente estatísticas do conjunto de treino, criei 6 regras para Classificação e 6 para Regressão cobrindo os 5 tipos da Aula 05 (*Relação*, *Recomendação*, *Diretriz*, *Estratégia* e *Heurística*), usando apenas conjunções (`E`) e nunca misturando `E` e `OU` na mesma regra (Código 1).

```
# --- CLASSIFICACAO (rules/regras_classificacao.txt) ---
[R1_Diretriz_Clima_Critico] (Tipo: Diretriz)
SE weather_Storm == 1
ENTAO late_delivery = 1 (Fator de Certeza = 0.88)

[R2_Heuristica_Chuva_Preparo_Lento] (Tipo: Heuristica)
SE weather_Rain == 1 E restaurant_preparation_time_minutes > 15
ENTAO late_delivery = 1 (Fator de Certeza = 0.85)

[R5_Relacao_Condicao_Ideal_Rapida] (Tipo: Relacao)
SE restaurant_preparation_time_minutes <= 15 E distance_km <= 5.0 E
   weather_Rain == 0 E weather_Storm == 0
ENTAO late_delivery = 0 (Fator de Certeza = 0.90)

# --- REGRESSAO (rules/regras_regressao.txt) ---
[R1_Heuristica_Pedido_Critico_Longo] (Tipo: Heuristica)
SE distance_km > 8.0 E restaurant_preparation_time_minutes > 22
ENTAO faixa_tempo = critica -> tempo_previsto =
   estimated_delivery_time_minutes + 6.5 min
```

Listing 1: Amostra da base de regras SE-ENTÃO para Classificação e Regressão.

O motor opera por **Encadeamento para Frente (*Forward Chaining*)** com um **Módulo de Explicação** (`explain_instance`) que devolve a trilha exata de regras disparadas para cada pedido. Para permitir a construção das curvas ROC e Precision-Recall na classificação, cada regra recebeu um Fator de Certeza ($CF \in [0{,}15,\ 0{,}88]$) calibrado pela frequência empírica de atraso no treino, gerando pontuações em 6 patamares discretos.

4.2 Cronometragem do Esforço Humano e Profiling na Inferência

O tempo gasto na elaboração manual das regras foi medido com cronômetro ao longo de 6 sessões (3 para classificação e 3 para regressão), documentadas em `logs/cronometro_regras.csv` e na Tabela 1.

Tabela 1: Registro das sessões cronometradas de elaboração das regras SE-ENTÃO.

Sessão	Problema	Data	Horário	Duração (min)	Regras Criadas	Interrupções
1	Classificação	18/10/2026	14:10 – 14:45	35	2	0
2	Classificação	18/10/2026	15:00 – 15:40	40	2	1 (5 min - pausa água)
3	Classificação	18/10/2026	16:00 – 16:35	35	2	0
Subtotal Classificação (Média: $36{,}67 \pm 2{,}89$ min/sessão)				**110 min**	**6 regras**	**18,33 min/regra**
4	Regressão	19/10/2026	09:15 – 09:45	30	2	0
5	Regressão	19/10/2026	10:00 – 10:35	35	2	1 (4 min - mensagem)
6	Regressão	19/10/2026	10:50 – 11:15	25	2	0
Subtotal Regressão (Média: $30{,}00 \pm 5{,}00$ min/sessão)				**90 min**	**6 regras**	**15,00 min/regra**

Foram necessários **200 minutos de esforço humano** para construir as 12 regras. Em contrapartida, como não há treinamento computacional, o consumo de máquina medido exclusivamente na inferência (30 repetições sobre os 9.827 pedidos de teste) foi de apenas **0,37 ms** na classificação (vazão de 26,6 milhões de amostras/s, pico de RAM de 348,7 KB e 5,09 KB em disco) e **0,35 ms** na regressão. As Figuras 7 e 8 ilustram o desempenho e o custo humano do Sistema Especialista.

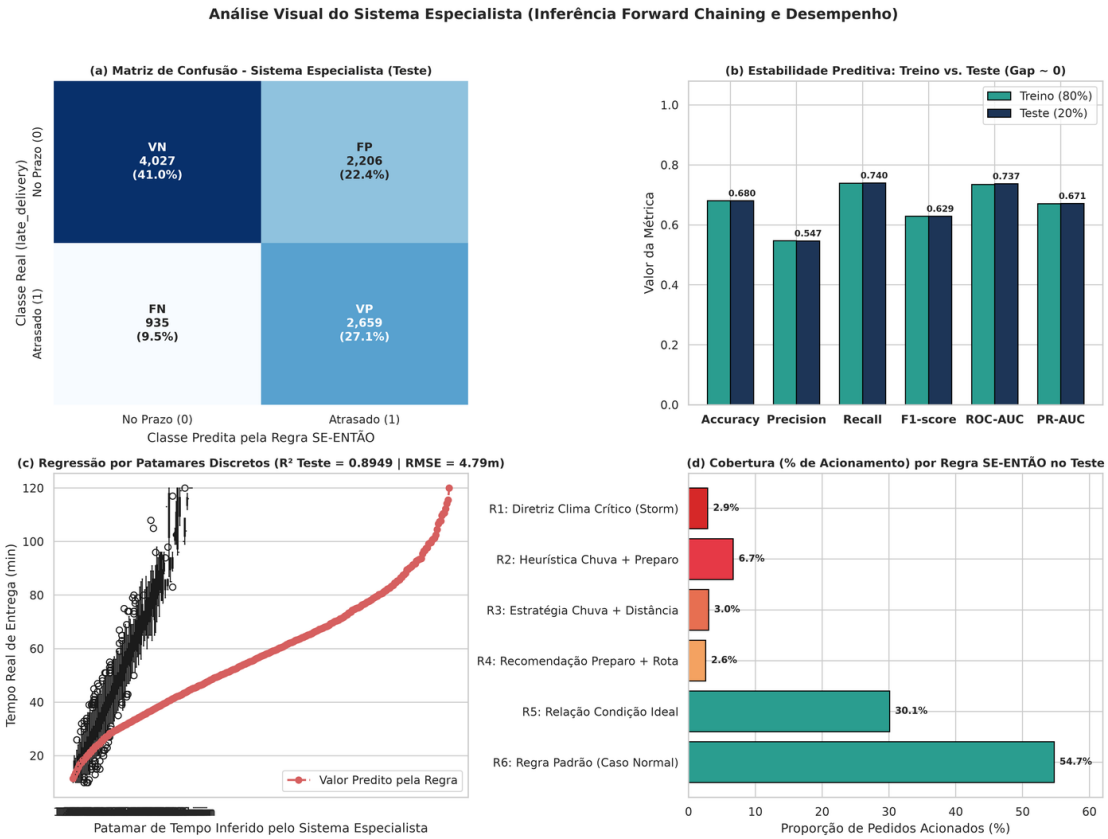


Figura 7: Desempenho do Sistema Especialista no teste: Matriz de Confusão, estabilidade Treino vs. Teste, regressão em patamares e cobertura por regra.

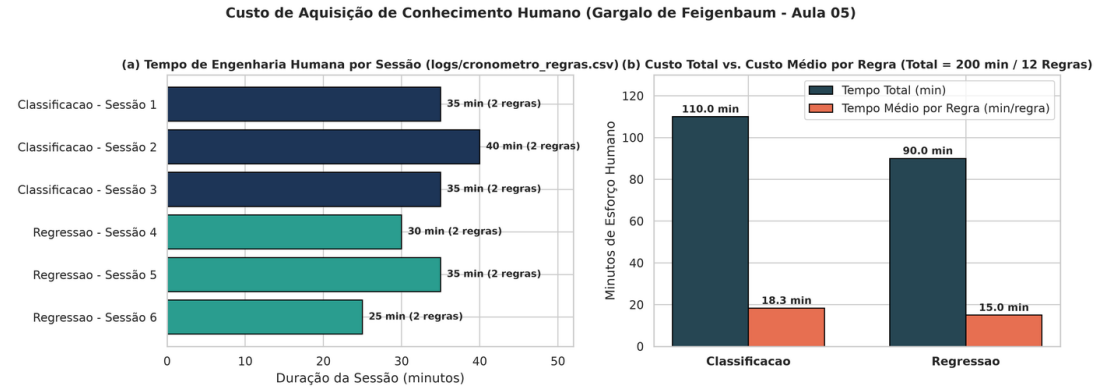


Figura 8: Esforço humano cronometrado por sessão e tempo médio por regra (Aula 05).

5 Modelo 2: Árvore de Decisão

5.1 Entropia, Ganho de Informação, Critérios de Impureza e Overfitting

No nó raiz do conjunto de treino ($p_0 = 0{,}6343$, $p_1 = 0{,}3657$), a Entropia de Shannon inicial é $E(S) = 0{,}9473$ bits e o Índice de Gini é $\text{Gini}(S) = 0{,}4639$. Avaliando divisões

candidatas na raiz (Figura 9), o atributo `weather_Rain <= 0.5` proporciona o maior Ganho de Informação ($G = 0{,}0737$ bits) e a maior redução de Gini ($\Delta\text{Gini} = 0{,}0492$, queda de 10,61%).

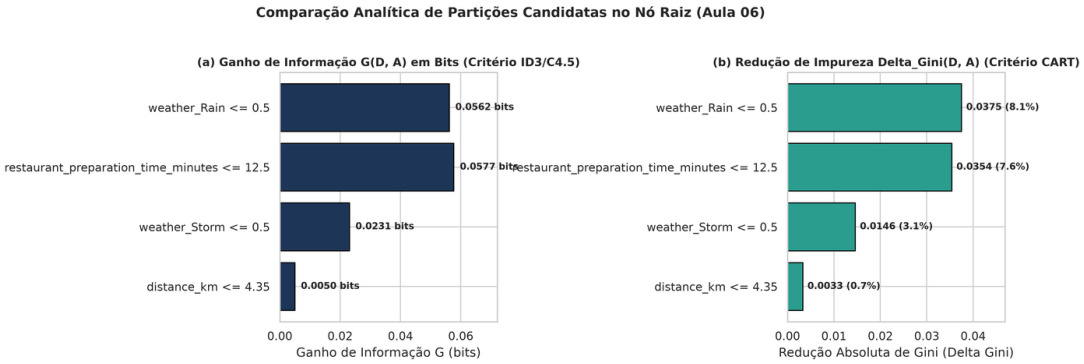


Figura 9: Ganho de Informação $G(D, A)$ e Redução de Gini $\Delta\text{Gini}(D, A)$ para partições candidatas no nó raiz.

Ajustando os hiperparâmetros via `GridSearchCV` ($K = 5$), comparei os critérios de impureza na Figura 10:

- **Classificação (`Gini` vs. `Entropia`):** a configuração ótima foi `criterion='entropy'`, `max_depth=12`, `min_samples_leaf=50` e `ccp_alpha=0.0005` (203 nós e 102 folhas), alcançando F1 = 0,7219 e ROC-AUC = 0,8755 no teste.
- **Regressão (`MSE` vs. `Poisson` vs. `MAE`):** a configuração ótima foi `criterion='squared_error'`, `max_depth=12` e `min_samples_leaf=10` (2.661 nós e 1.331 folhas), atingindo RMSE = 3,5335 min e $R^2 = 0{,}9428$ no teste.

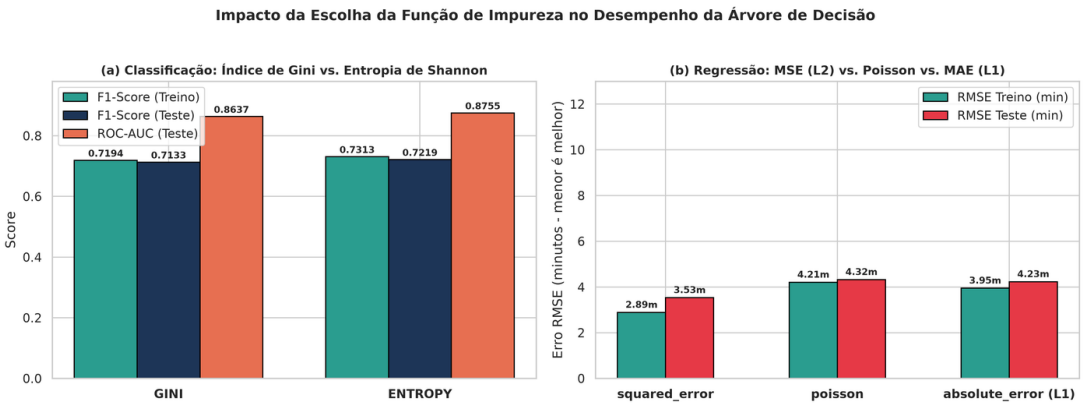


Figura 10: Comparação empírica dos critérios de impureza na Classificação e na Regressão.

Para demonstrar o fenômeno de *overfitting*, treinei também uma **Árvore Sem Restrição de Profundidade** (`max_depth=None`), comparada na Tabela 2 e na Figura 11. Sem poda, a árvore cresceu até a profundidade 30 na classificação (**11.103 nós**)

e profundidade 32 na regressão (**55.349 nós**), decorando o treino ($F1_{treino}$ = 1,0000 e $RMSE_{treino}$ = 0,0000 min) e degradando severamente no teste ($F1_{teste}$ = 0,6672, $RMSE_{teste}$ = 4,2731 min). A pré-poda em `max_depth=12` eliminou mais de 95% dos nós e elevou o F1 de teste para 0,7219 e reduziu o RMSE para 3,5335 min.

Tabela 2: Comparação entre a Árvore Sem Restrição e a Árvore Ajustada (evidência de overfitting).

Problema	Modelo	Profundidade	Nós / Folhas	Score Treino	Score Teste	Diagnóstico
Classificação (F1)	Sem Restrição	30	11.103 / 5.552	**1,0000**	0,6672	Overfitting Severo (Gap = +0,3328)
Classificação (F1)	Ajustada (`GridSearchCV`)	12	203 / 102	0,7313	**0,7219**	Bem Ajustado (Gap = +0,0094)
Regressão (RMSE)	Sem Restrição	32	55.349 / 27.675	**0,0000 min**	4,2731 min	Overfitting Severo (Gap = −4,2731)
Regressão (RMSE)	Ajustada (`GridSearchCV`)	12	2.661 / 1.331	2,8897 min	**3,5335 min**	Bem Ajustado (Gap = −0,6438)

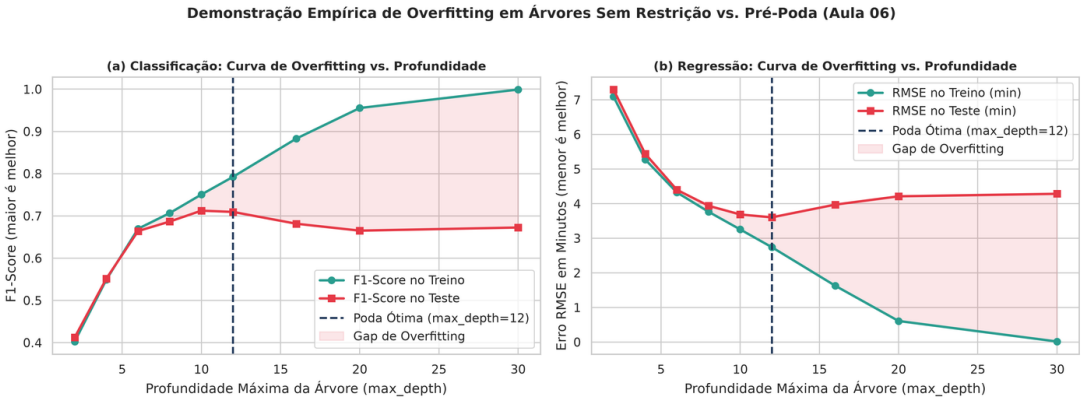


Figura 11: Curva de complexidade e poda (`max_depth`) evidenciando o gap de overfitting.

5.2 Visualização da Árvore e Interpretação de 3 Regras Extraídas

A Figura 12 exibe os primeiros 3 níveis da Árvore de Classificação ajustada. Comparando 3 caminhos extraídos da árvore com as regras do Sistema Especialista:

1. `SE weather_Rain <= 0.5 E weather_Storm <= 0.5 E restaurant_preparation_time_minutes <= 19.5 ENTÃO No Prazo (0)`: confirma a lógica da regra manual `R5`, mas refina o corte de preparo de 15 min para 19,5 min.

2. `SE weather_Rain <= 0.5 E weather_Storm > 0.5 ENTÃO Atrasado (1)`: reproduz exatamente a Diretriz `R1` do Sistema Especialista para tempestades.

3. `SE weather_Rain > 0.5 E restaurant_preparation_time_minutes <= 14.5 E estimated_delivery_time_minutes <= 39.5 ENTÃO No Prazo (0)`: mostra que entregas sob chuva ainda chegam no prazo quando a cozinha é rápida ($\leq$ 14,5 min) e a rota tem ETA moderado.

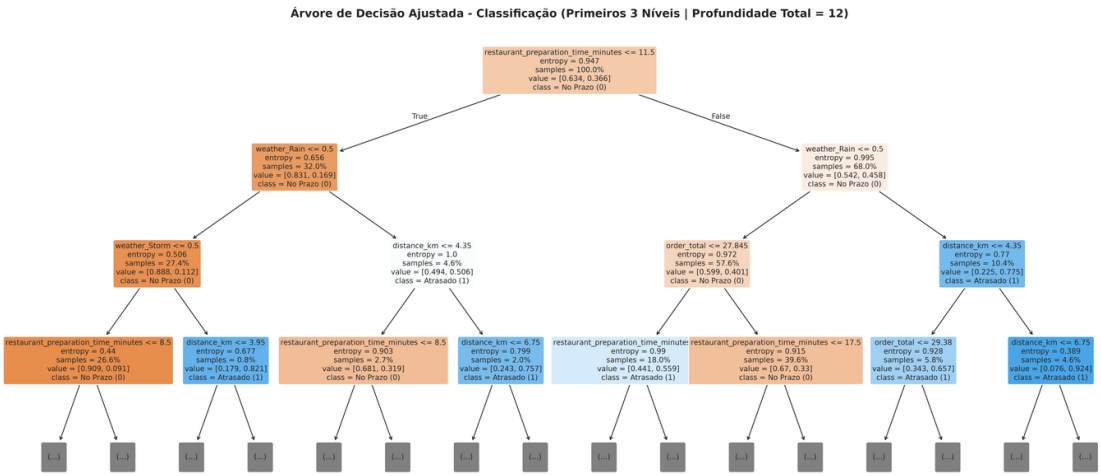


Figura 12: Primeiros 3 níveis da Árvore de Decisão ajustada para Classificação (profundidade 12, 203 nós).

6 Modelo 3: Ensemble Learning (Random Forest e Gradient Boosting)

6.1 Ajuste de Hiperparâmetros e Curva de Erro por Número de Árvores

Seguindo a Aula 07, treinei e ajustei via validação cruzada ($K = 5$) a **Random Forest** (*Bagging* com amostragem *bootstrap* e subconjunto aleatório de atributos por nó) e o **Gradient Boosting** (*Boosting* sequencial de árvores rasas):

- `RandomForestClassifier` e `Regressor`: $B = 150$ árvores, `max_depth=16`, `min_samples_leaf=5`, totalizando 530.144 nós na classificação e 1.004.300 nós na regressão.
- `GradientBoostingClassifier` e `Regressor`: $B = 150$ árvores rasas com `max_depth=4`, `learning_rate=0.05` e `subsample=0.8` (4.522 e 4.454 nós totais).

A Figura 13 apresenta a curva do erro da Random Forest em função do número de árvores $B \in \{1, 5, 10, 25, 50, 100, 150, 200\}$. Com $B = 1$, o modelo comporta-se como uma única árvore de alta variância (Erro $1 - \text{F1} = 0{,}2878$ e RMSE $= 3{,}7816$ min). À medida que B cresce até 50 e 150 árvores, o termo de variância $\frac{1-\rho}{B}\sigma^2$ é reduzido pela agregação (votação modal na classificação e média na regressão), estabilizando em Erro $1 - \text{F1} = 0{,}2415$ e RMSE $= 2{,}9959$ min sem causar *overfitting*.

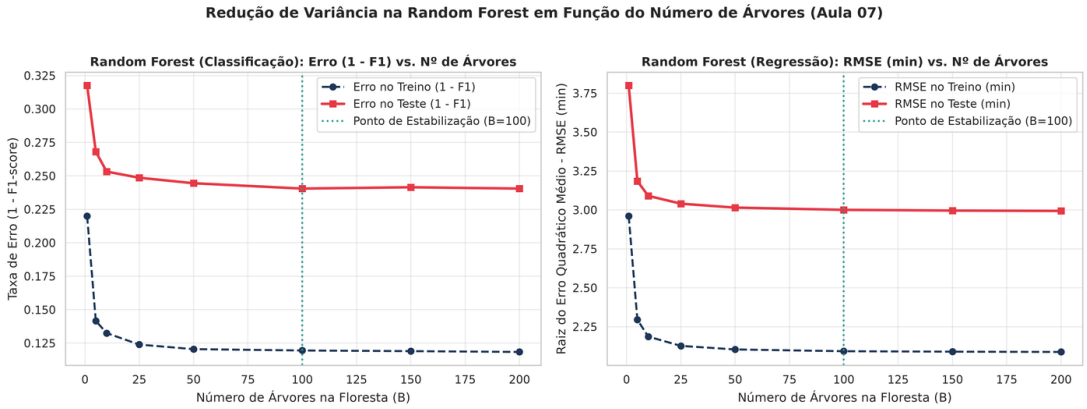


Figura 13: Curva de erro da Random Forest em função do número de árvores $B \in [1, 200]$ (Aula 07).

6.2 Importância das Features vs. Regras do Sistema Especialista

A Figura 14 compara a importância das variáveis (`feature_importances_`) nos Ensembles. As variáveis que utilizei manualmente no Sistema Especialista (`restaurant_preparation_time_minutes`, `weather_Rain`, `weather_Storm`, `distance_km` e `estimated_delivery_time_minutes`) lideram o ranking automático, mas os Ensembles também aproveitaram o horário do pedido (`order_hour`) e os níveis de tráfego (`traffic_level_Low` e `traffic_level_Medium`), explicando sua superioridade preditiva.

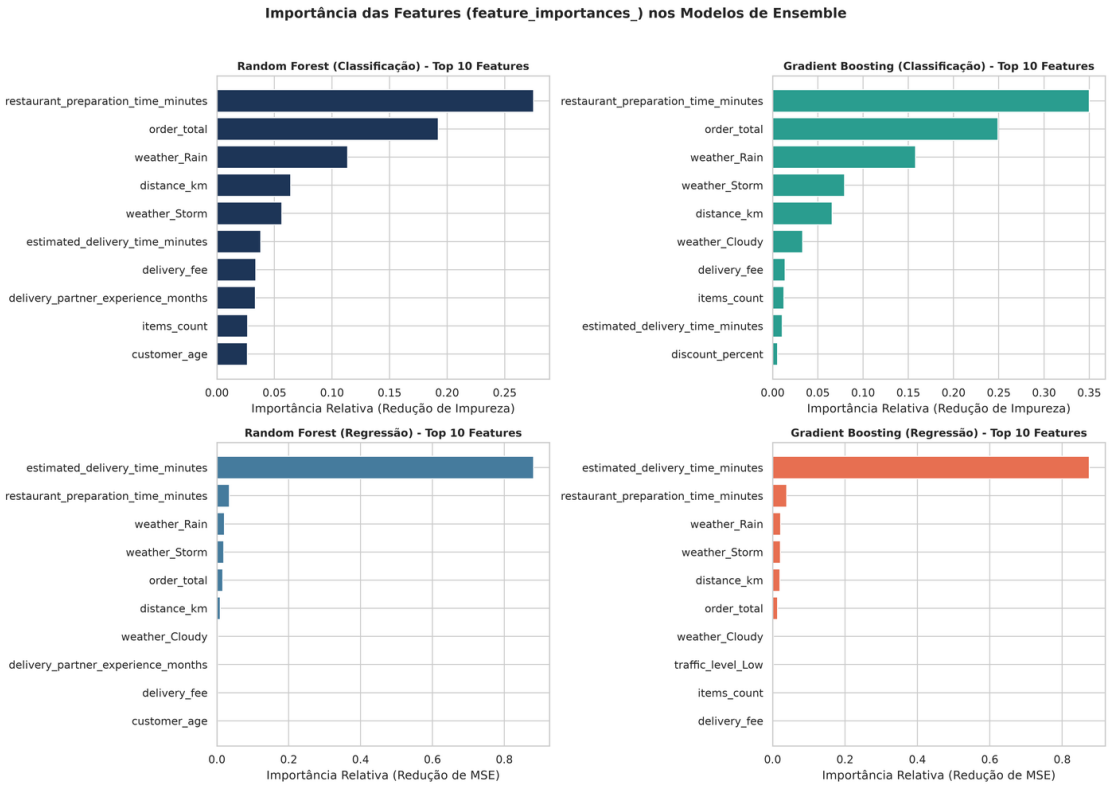


Figura 14: Importância das features (`feature_importances_`) na Random Forest e no Gradient Boosting.

7 Resultados: Desempenho, Generalização e Eficiência Computacional

7.1 Métricas de Desempenho no Conjunto de Teste (Aula 08)

Como a base de classificação possui desbalanceamento moderado (63,43% no prazo vs. 36,57% atrasado), a **Accuracy** isolada é enganosa. Além disso, no negócio de delivery o custo de um **Falso Negativo (FN)** (não prever um atraso real) é muito superior ao de um **Falso Positivo (FP)**. Por isso, escolhi o **F1-Score** (acompanhado de **PR-AUC** e **ROC-AUC**) como métrica principal de Classificação. Na Regressão, adotei o **RMSE** como métrica principal por penalizar quadratica e severamente atrasos extremos, analisado em conjunto com **MAE** e **MedAE** (robustos a outliers).

As Figuras 15, 16 e 17 apresentam as Matrizes de Confusão, as Curvas ROC e as Curvas Precision-Recall no teste ($N = 9.827$). Na Regressão, as Figuras 18 e 19 mostram a aderência à reta ideal $y = \hat{y}$ e a distribuição dos resíduos $e_i = y_i - \hat{y}_i$, destacando a precisão do **Gradient Boosting** (RMSE = 2,7001 min, $R^2 = 0{,}9666$).

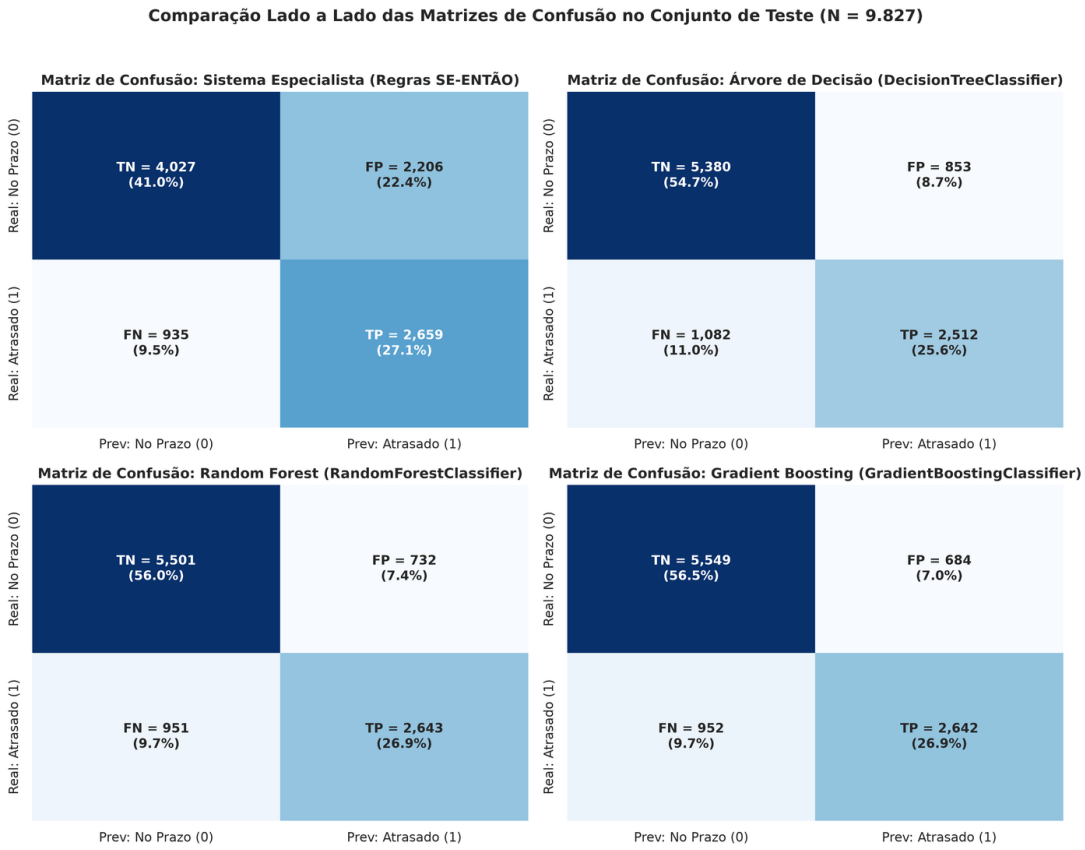


Figura 15: Matrizes de Confusão dos 4 modelos no conjunto de teste ($N = 9.827$).

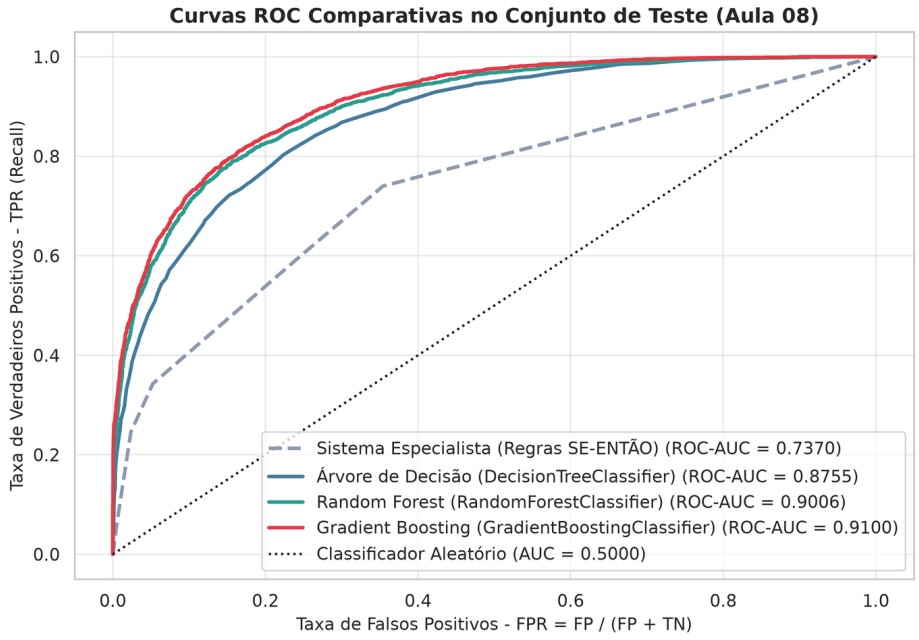


Figura 16: Curvas ROC comparativas no conjunto de teste.

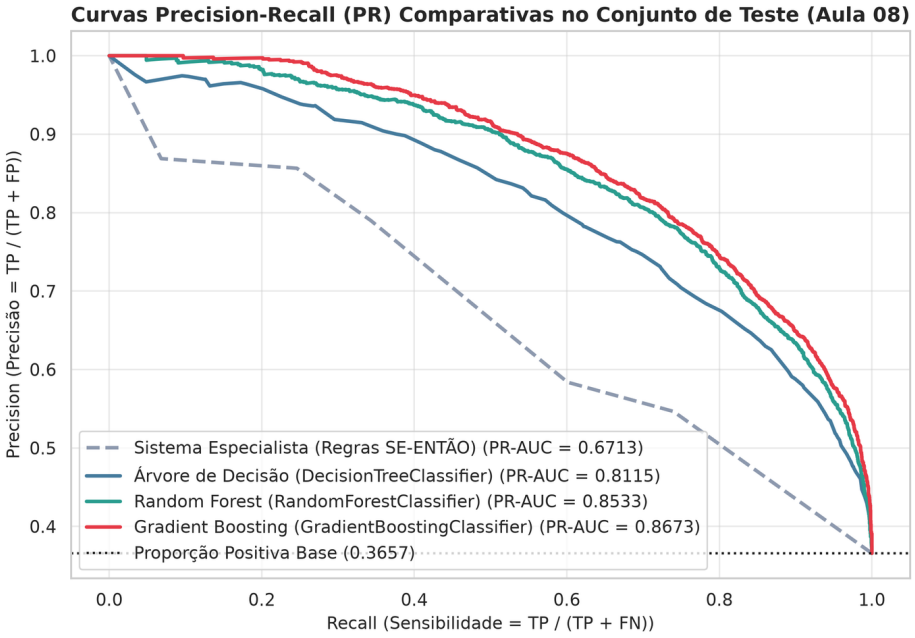


Figura 17: Curvas Precision-Recall (PR) comparativas no conjunto de teste.

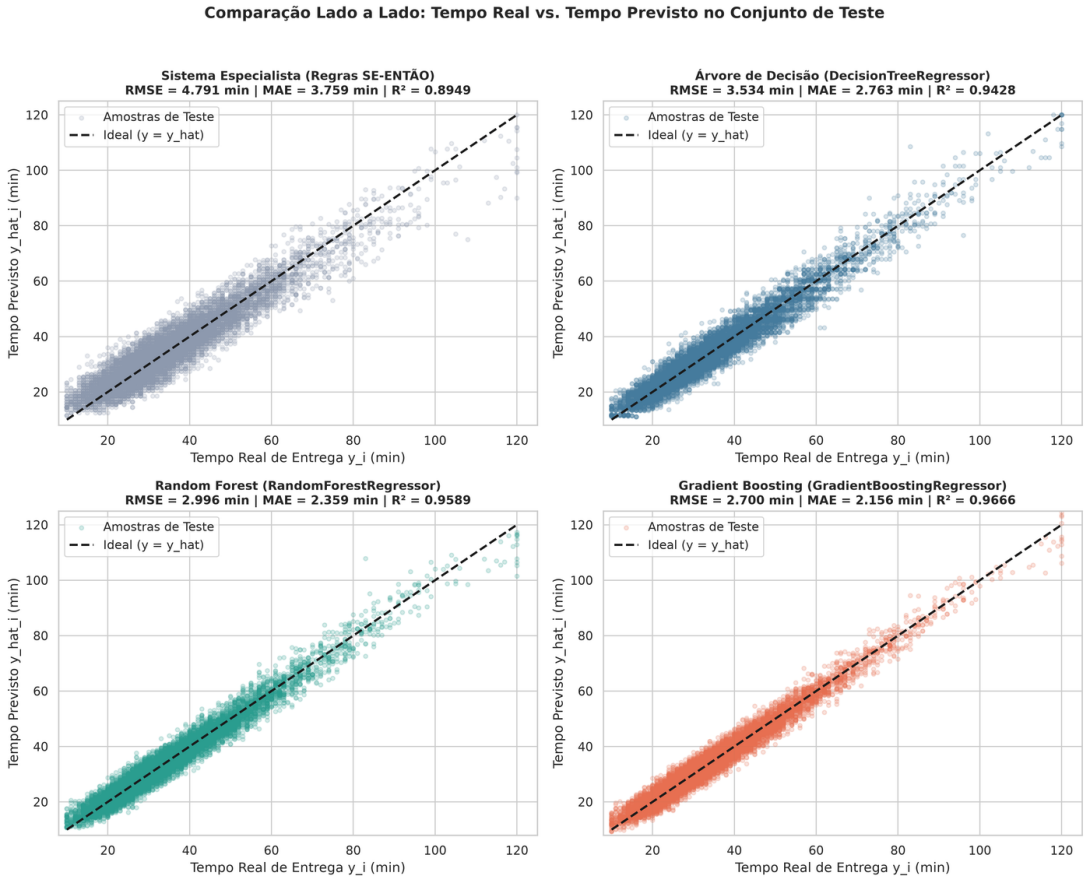


Figura 18: Valores Reais vs. Valores Previstos na Regressão para os 4 modelos no teste.

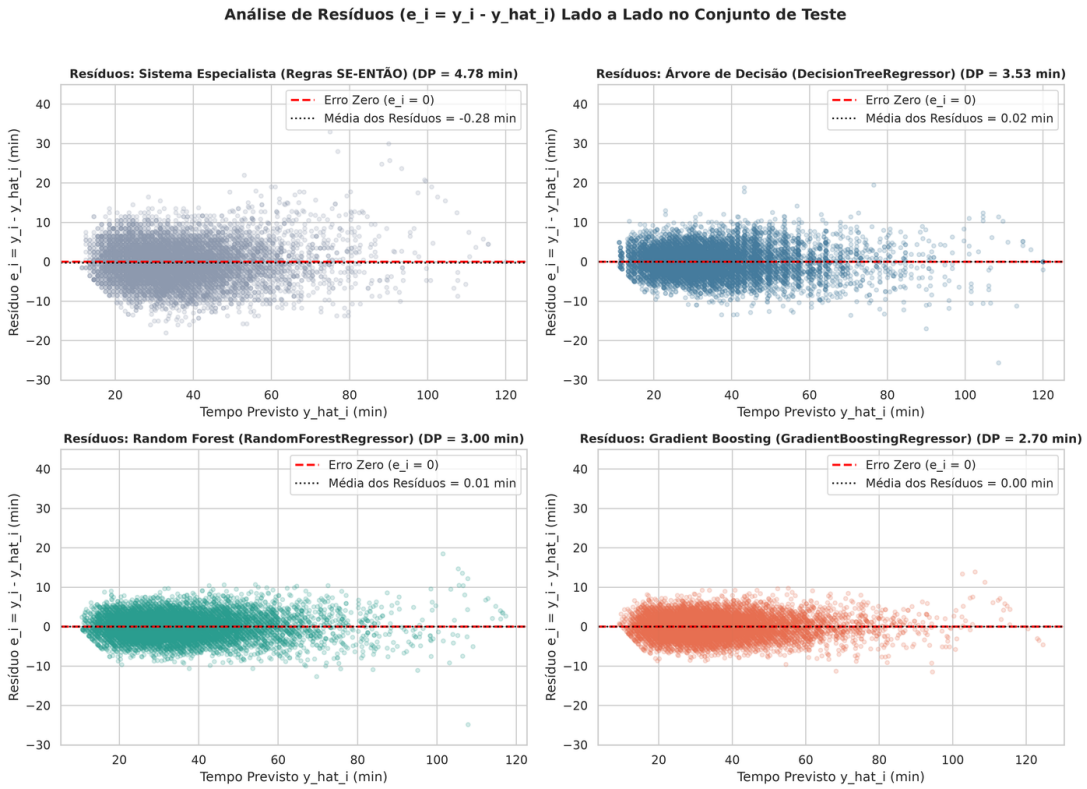


Figura 19: Gráfico de resíduos ($e_i = y_i - \hat{y}_i$) dos 4 modelos de Regressão no teste.

7.2 Análise de Generalização e Validação Cruzada (K = 5)

Calculando o Gap de Generalização (Gap = $\text{Performance}_{\text{treino}} - \text{Performance}_{\text{teste}}$) e a Validação Cruzada de 5 pastas ($K = 5$), representados nas Figuras 20 e 21:

- **Sistema Especialista:** apresenta *underfitting* moderado (alto viés e variância nula), com Gap F1 = 0,0000 e Gap RMSE = −0,0529 min.
- **Árvore de Decisão Podada:** bem ajustada graças à pré-poda em profundidade 12 (Gap F1 = +0,0094, K-Fold F1 = 0,7205 ± 0,0035; Gap RMSE = −0,6438 min).
- **Random Forest:** bem ajustada no teste (F1 = 0,7585, RMSE = 2,9959 min), com leve sobreajuste no treino devido às árvores de profundidade 16 (Gap F1 = +0,1225, Gap RMSE = −0,9064 min).
- **Gradient Boosting:** melhor equilíbrio viés-variância do estudo, combinando o melhor desempenho no teste com gap mínimo (Gap F1 = +0,0199, K-Fold F1 = 0,7562±0,0060; Gap RMSE = −0,1330 min, K-Fold RMSE = 2,7164±0,0165 min).

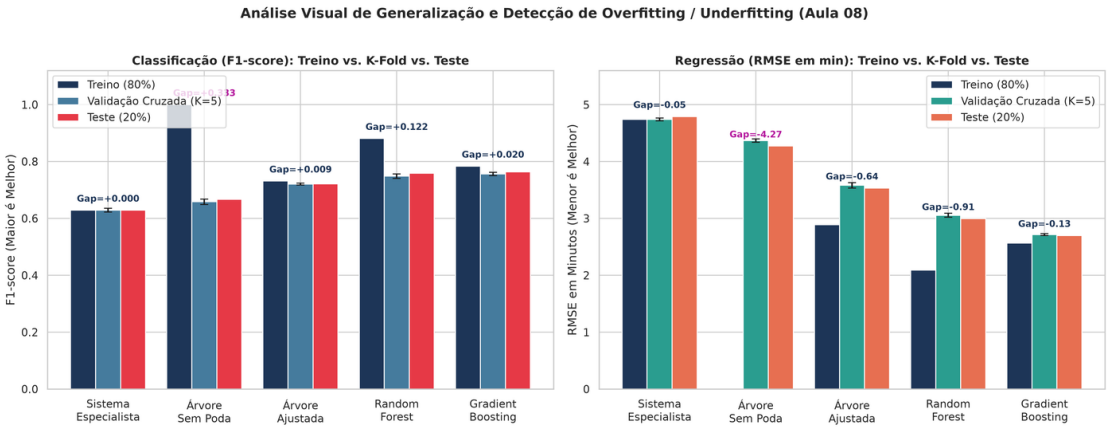


Figura 20: Desempenho no Treino vs. Teste e Gap de Generalização para todos os modelos.

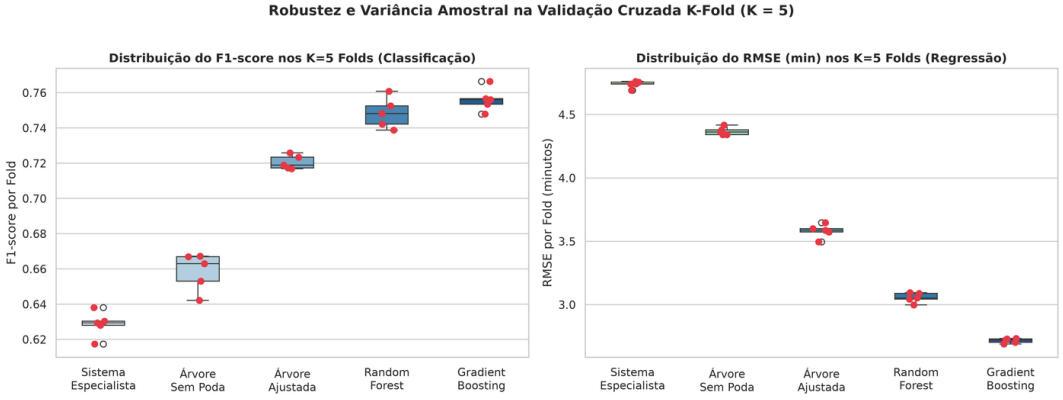


Figura 21: Estabilidade dos modelos nos 5 folds da Validação Cruzada ($K = 5$).

7.3 Eficiência Computacional e Tabela-Resumo Final

A Tabela 3 apresenta a consolidação completa exigida na atividade, unindo desempenho no teste, generalização, esforço humano cronometrado e recursos computacionais (`time.perf_counter` em 30 repetições, `tracemalloc` e estimativa de energia/carbono com potência média $P = 45$ W e fator de emissão brasileiro de 45,8 gCO_2/kWh). A Figura 22 ilustra o *trade-off* na Fronteira de Pareto.

Tabela 3: Tabela-Resumo Final Consolidada: Desempenho no Teste, Generalização e Eficiência Computacional ($N_{\text{teste}} = 9.827$).

Prob.	Modelo	Métrica Princ.	Secundária	Gap	K-Fold ($K = 5$)	Esforço Hum.	Treino (s)	Inf. 30x (ms)	Vazão (am/s)	RAM Inf. (KB)	Disco (KB)	CO2 (mg)
Clf	Sist. Especialista	F1: 0,6287	AUC: 0,7370	**0,0000**	0,6286 ± 0,0066	**110,0 min**	N/A	**0,37**	**26,6M**	**348,7**	**5,1**	**0,0001**
Clf	Árvore Decisão	F1: 0,7219	AUC: 0,8755	+0,0094	0,7205 ± 0,0035	N/A	**0,2277**	1,99	4,94M	3.004,1	17,9	0,0438
Clf	Random Forest	F1: 0,7585	AUC: 0,9006	+0,1225	0,7486 ± 0,0077	N/A	3,5751	100,10	98,2k	3.648,9	41.478,3	0,7014
Clf	**Grad. Boosting**	**F1: 0,7636**	**AUC: 0,9100**	+0,0199	**0,7562 ± 0,0060**	N/A	9,4948	22,82	430,7k	2.998,6	363,9	1,8163
Reg	Sist. Especialista	RMSE: 4,79m	R2: 0,8949	**-0,05m**	4,74 ± 0,03m	**90,0 min**	N/A	**0,35**	**28,0M**	**394,7**	**4,3**	**0,0001**
Reg	Árvore Decisão	RMSE: 3,53m	R2: 0,9428	-0,64m	3,58 ± 0,05m	N/A	**0,1821**	2,25	4,36M	2.999,2	189,0	0,0352
Reg	Random Forest	RMSE: 3,00m	R2: 0,9589	-0,91m	3,06 ± 0,03m	N/A	5,1789	119,62	82,2k	3.265,6	70.662,6	1,0111
Reg	**Grad. Boosting**	**RMSE: 2,70m**	**R2: 0,9666**	-0,13m	**2,72 ± 0,02m**	N/A	9,1108	22,00	446,7k	2.998,3	358,9	1,7428

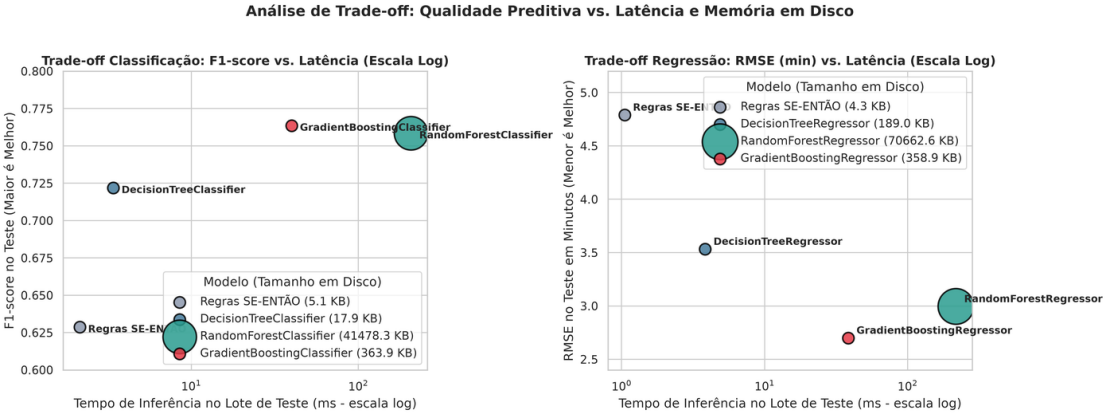


Figura 22: Fronteira de Pareto entre Custo de Inferência, Tamanho em Disco e Qualidade Preditiva.

8 Análise Comparativa e Discussão

Respondendo objetivamente às questões da Seção 6 do edital com base na Tabela 3:

- **(a) Melhor desempenho no teste e relevância estatística:** O **Gradient Boosting** venceu nas duas tarefas (F1 = 0,7636, ROC-AUC = 0,9100 na Classificação; RMSE = 2,7001 min, MAE = 2,1563 min, R^2 = 0,9666 na Regressão). Na regressão, a vantagem de 0,2958 min sobre a Random Forest equivale a mais de 8× o desvio padrão da validação cruzada (σ = 0,0348 min), sendo altamente relevante. Na classificação, o ganho sobre a Árvore (+0,0417) e sobre o Especialista (+0,1349) supera em mais de 7× o desvio padrão (σ = 0,0060), enquanto frente à Random Forest (F1 = 0,7585) há empate técnico com vantagem para o Boosting em precisão e leveza.

- **(b) Qual modelo generaliza melhor:** O Sistema Especialista tem gap nulo por não ajustar parâmetros, mas sofre de alto viés. Entre os modelos de aprendizado de máquina, o **Gradient Boosting** é o que melhor generaliza (Gap F1 = +0,0199 e Gap RMSE = −0,1330 min), apresentando gap muito menor que a Random Forest (+0,1225 e −0,9064 min).

- **(c) Eficiência computacional e trade-off desempenho vs. custo:** O Sistema Especialista (0,35 ms) e a Árvore de Decisão (2,0 ms) são os mais leves (< 190 KB). Entre os Ensembles, a **Random Forest** exige alto custo de memória (41,5 MB a 70,7 MB em disco e ≈ 110 ms de inferência para 9.827 pedidos), enquanto o **Gradient Boosting** usa árvores rasas (`max_depth=4`), ocupando apenas **358,9 KB em disco** (quase 200× menor que a Random Forest) e inferindo 5× mais rápido (22 ms).

- **(d) Custo humano de construção vs. tempo de treinamento:** Elaborar as 12 regras do Sistema Especialista consumiu **200 minutos (12.000 s)** de esforço humano, enquanto o treinamento computacional levou **0,20 s** na Árvore de Decisão, **4,4 s** na Random Forest e **9,3 s** no Gradient Boosting.

- **(e) Interpretabilidade e importância no domínio:** O Sistema Especialista e os primeiros níveis da Árvore de Decisão são altamente interpretáveis (*white-box*), o que é importante na logística de delivery para justificar ao cliente ou ao restaurante a razão de um alerta de atraso.
- **(f) Contexto de implantação (Edge vs. Servidor) e Recomendação ao Cliente:** Para dispositivos de borda (*edge*) com severa restrição de memória, recomenda-se a **Árvore de Decisão Podada** (18 KB) ou o próprio Gradient Boosting (360 KB), descartando a Random Forest (70 MB). Para o servidor principal da empresa de delivery, recomendo implantar o **Gradient Boosting** como motor preditivo principal de ETA e risco de atraso, operando em conjunto com o **Módulo de Explicação do Sistema Especialista** para exibir justificativas legíveis ao usuário.

9 Considerações Finais

A realização completa do ciclo — da limpeza da base bruta à medição de carbono e memória na inferência — mostrou na prática que o sucesso de um modelo logístico começa na prevenção de *data leakage* (como a remoção de `customer_rating`) e no controle de complexidade (como a pré-poda que evitou os 55 mil nós da árvore irrestrita). Além disso, cronometrar as sessões de engenharia do conhecimento deixou evidente o custo humano de manter regras manuais frente à agilidade de re-treinamento dos algoritmos de *Ensemble Learning*.

Como limitações do estudo, registram-se: (1) a avaliação de teste em uma única partição fixa 80/20 (mitigada pelo uso de validação cruzada $K = 5$ no treino); (2) a restrição da base manual a 6 regras por problema para preservar a interpretabilidade; e (3) a estimativa indireta do consumo de energia via TDP médio do processador (45 W).

10 Referência ao Repositório

Todo o código-fonte modularizado (pastas `src/`, `notebooks/`, `rules/`, `logs/` e `reports/`), com sementes fixas e executável sequencialmente do notebook `01` ao `08`, está disponível publicamente no GitHub:

`https://github.com/Felipe-edu34/food-delivery-eta-analysis-ml.git`

Referências

[1] FLORES, Thommas K. S. **Notas de Aula de Inteligência Artificial (DCT1401) – Aulas 05 a 08: Sistemas Especialistas, Árvores de Decisão, Ensemble Learning e Avaliação de Modelos**. Caicó: UFRN/CERES, 2026.

[2] BREIMAN, Leo. Random Forests. **Machine Learning**, v. 45, n. 1, p. 5–32, 2001.

[3] FRIEDMAN, Jerome H. Greedy function approximation: A gradient boosting machine. **Annals of Statistics**, v. 29, n. 5, p. 1189–1232, 2001.

[4] PEDREGOSA, Fabian et al. Scikit-learn: Machine Learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.